P3481R5

`std::execution::bulk()` issues

Published Proposal, 2025-06-18

This version:

http://wg21.link/P3481R5

Authors:

Lucian Radu Teodorescu (Garmin)

Ruslan Arutyunyan (Intel)

Lewis Baker

Mark Hoemmen (NVIDIA)

Audience:

LWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

This paper explores the issues with `std::execution::bulk` algorithm and proposes a way to address them, making the API better for the users.

Table of Contents

1. Changes history

1.1. R5

- Apply LWG review feedback from Bulgaria, 2025

1.2. R4

- Improve wording.

- Add execution policy to `bulk_unchunked`.
- remove extra check in `bulk_unchunked` for concurrent schedulers

1.3. R3

- Improve wording.

1.4. R2

- Incorporate to the feedback from SG1 review, Austria, 2025.
  - Incorporate the changes from [P3564R0]: Make calling un-customized `bulk_unchunked` ill-formed on a scheduler with concurrent forward progress.
  - Remove `bulk_unchunked`'s execution policy parameter.
  - Specify an error for the "lack of resources" case.

1.5. R1

- Incorporate to the feedback from SG1 review, Poland, 2024.
- Add wording section.

2. Motivation

When [P2300R10] was merged into the working draft, a subsequent review of the github issue tracker discovered several outstanding issues relating to the use of the `bulk` algorithm that was in [P2300R10] which were not addressed during the LEWG review. These issues mostly account for better expressing what `bulk` can and cannot do.

For most algorithms defined in [P2300R10], the default implementation typically describes the desired behavior. However, this is not the case for `bulk`.

The expectation for `bulk` is that it will be customized for most use cases. The default implementation is a common-denominator solution that is not necessarily optimal for the majority of scenarios.

The description in [exec.bulk] specifies the default implementation but does not outline any requirements for customizations. This creates uncertainty for users regarding what they can expect when in-

voking `bulk`.

This paper addresses this issue by specifying what is permissible in a customization of `bulk`.

A second issue this paper seeks to resolve is the lack of an execution policy for the provided functor. For example, imagine that we have scheduler that supports `par_unseq` execution policy only for parallelism (certain GPU schedulers) and it wants to customize `bulk`. If the functor is unsuitable for the `par_unseq` execution policy, there is currently no mechanism for users to express this with the current API shape.

A third issue this paper addresses is the absence of chunking in the default implementation of `bulk`. Invoking the functor for every element in a range can lead to performance challenges in certain use cases.

§ 2.1. Allowed behavior for `bulk` customizations

From the specification of bulk in [exec.bulk], if we have customized `bulk`, the following are unclear:

- Can `bulk` invoke the given functor concurrently?
- Can `bulk` create decayed copies of the given functor?
- Can `bulk` create decayed copies of the values produced by the previous sender?
- Can `bulk` handle cancellation within the algorithm?
- How should `bulk` respond to exceptions thrown by the given functor?

§ 2.2. Execution policy for the given functor

Similar to `for_each` ([alg.foreach]) users might expect the `bulk` algorithm to take an execution policy as argument. The execution policy would define the execution capabilities of the given function.

There are two main reasons for wanting an execution policy to accompany the function passed to `bulk`:

1. To better tailor the algorithm to the available hardware.
2. To improve usage in generic code.

Currently, we have hardware that supports both `par` and `par_unseq` execution policy semantics, as well as hardware that only supports `par_unseq`. Standardizing the API without execution policies

implies that the language chooses a default execution policy for `bulk`. This decision could favor certain hardware vendors over others.

From a generic programming perspective, consider a scenario where a user implements a function like the following:

```
void process(std::vector<int>& data, auto std::invocable<int> f) {
  // call `f` for each element in `data`; try using `bulk`.
  // can we call `f` concurrently?
}
```

In the body of the generic function process, we do not know the constraints imposed on `f`. Can we assume that `f` can be invoked with the `par_unseq` execution policy? Can it be invoked with the `par` execution policy? Or should we default to the `seq` execution policy?

Since there is no way to infer the execution policy that should apply to `f`, the implementation of process must be conservative and default to the `seq` execution policy. However, in most cases, the `par` or `par_unseq` execution policy would likely be more appropriate.

§ 2.3. Chunking

The current specification of `bulk` ([exec.bulk]) does not support chunking. This limitation presents a performance issue for certain use cases.

If two iterations can potentially be executed together more efficiently than in isolation, chunking would provide a performance benefit.

Let us take an example:

```
std::atomic<std::uint32_t> sum{0};
std::vector<std::uint32_t> data;
ex::sender auto s = ex::bulk(ex::just(), std::execution::par, 100'000,
  [&sum,&data](std::uint32_t idx) {
    sum.fetch_add(data[idx]);
  });
```

In this example, we perform 100,000 atomic operations. A more efficient implementation would allow a single atomic operation for each chunk of work, enabling each chunk to perform local summation. This approach might look like:

```
std::atomic<std::uint32_t> sum{0};
ex::sender auto s = ex::bulk_chunked(ex::just(), std::execution::par, 100'0
  [&sum,&data](std::uint32_t begin, std::uint32_t end) {
    std::uint32_t partial_sum = 0;
    while (begin != end) {
      partial_sum += data[idx];
    }
    sum.fetch_add(partial_sum);
  });
```

Other similar examples exist where grouping operations together can improve performance. This is particularly important when the functor passed to `bulk` cannot be inlined or when the functor is expensive to invoke.

§ 3. API

Define `bulk` to match the following API:

```
// NEW: algorithm to be used as a default basis operation
template<execution::sender Predecessor,
         typename ExecutionPolicy,
         std::integral Size,
         std::invocable<Size, Size, values-sent-by(Predecessor)...> Func
>
execution::sender auto bulk_chunked(Predecessor pred,
                                    ExecutionPolicy&& pol,
                                    Size size,
                                    Func f);

// NEW: algorithm to be used when one execution agent per iteration is des
template<execution::sender Predecessor,
         typename ExecutionPolicy,
         std::integral Size,
         std::invocable<Size, values-sent-by(Predecessor)...> Func
>
execution::sender auto bulk_unchunked(Predecessor pred,
```

```
                                        ExecutionPolicy&& pol,
                                        Size size,
                                        Func f);

template<execution::sender Predecessor,
         typename ExecutionPolicy,
         std::integral Size,
         std::invocable<Size, values-sent-by(Predecessor)...> Func
>
execution::sender auto bulk(Predecessor pred,
                            ExecutionPolicy&& pol, // NEW
                            Size size,
                            Func f) {
    // Default implementation
    return bulk_chunked(
        std::move(pred),
        std::forward<ExecutionPolicy>(pol),
        size,
        [func=std::move(func)]<typename... Vs>(
            Size begin, Size end, Vs&&... vs)
            noexcept(std::is_nothrow_invocable_v<Func, Size, Vs&&...>) {
            while (begin != end) {
                f(begin++, std::forward<Vs>(vs)...);
            }
        });
}
```

§ 4. Design discussions

§ 4.1. Use chunked version as a basis operation

To address the performance problem described in the motivation, we propose to add a chunked version for `bulk`. This allows implementations to process the iteration space in chunks, and take advantage of the locality of the chunk processing. This is useful when publishing the effects of the functor may be expensive (the example from the motivation) and when the given functor cannot be inlined.

The implementation of `bulk_chunked` can use dynamically sized chunks that adapt to the workload at hand. For example, computing the results of a Mandelbrot fractal on a line is an unbalanced process. Some values can be computed very fast, while others take longer to compute.

Passing a range as parameters to the functor passed to `bulk_chunked` is similar to the way Intel TBB's `parallel_for` functions (see [parallel_for]).

An implementation of `bulk` can be easily built on top of `bulk_chunked` without losing performance (as shown in the Proposal section).

§ 4.2. Also define an unchunked version

[P3564R0] explains the necessity of having a version of `bulk` that executes each loop iteration on a distinct execution agent:

1. For schedulers that promise concurrent forward progress, users need a way to take advantage of concurrent forward progress. Since `bulk` execution can do chunking, it cannot promise anything stronger than parallel forward progress.
2. Even for schedulers with a weaker forward progress guarantee, users sometimes have performance reasons to control the distribution of loop iterations to execution agents.

Revisions R2 and R3 originally incorporated this design suggestion from [P3564R0] into this paper and name this API `bulk_unchunked`. After further discussions in Sofia 2025, we realized that these two directions do not overlap well, and that it likely deserve separate APIs for the two design aspects. We decided that we can have `bulk_unchunked` as an algorithm to be used when the user wants to execute each iteration on a distinct execution agent, and a new algorithm `bulk_concurrent` that explicitly deals with concurrent forward progress guarantee.

This paper proposes `bulk_unchunked`, but does not propose `bulk_concurrent`, leaving this for future work.

§ 4.3. Using execution policies

As discussed in the § 2 Motivation section, not having the possibility to specify execution policies for the `bulk` algorithm is a downside. On the one hand, defaulting to any policy other than `seq` might lead to deadlocks. On the other hand, defaulting to `seq` would not match users' expectations that `bulk` would take advantage of parallel execution resources. Thus, `bulk` has to have an execution policy parameter.

One criticism of this solution is that each invocation of `bulk` needs to contain an extra parameter that is typically verbose to type.

The idea considered by the authors to solve it is to provide versions of the algorithms that have the execution policies already baked in. Something like:

```
auto bulk_seq(auto prev, auto size, auto f);
auto bulk_unseq(auto prev, auto size, auto f);
auto bulk_par(auto prev, auto size, auto f);
auto bulk_par_unseq(auto prev, auto size, auto f);

auto bulk_chunked_seq(auto prev, auto size, auto f);
auto bulk_chunked_unseq(auto prev, auto size, auto f);
auto bulk_chunked_par(auto prev, auto size, auto f);
auto bulk_chunked_par_unseq(auto prev, auto size, auto f);
```

We dropped this idea, as this isn't scalable.

On the other hand, it's quite easy to write a thin wrapper on top of `bulk` / `bulk_chunked` / `bulk_unchunked` on the user side that calls the algorithm with the execution policy of choice. Something like:

```
auto user_bulk_par(auto prev, auto size, auto f) {
  return std::execution::bulk(std::execution::par, prev, size, f);
}
```

§ 4.4. Conflicting execution policies

For [P2500R2] design we previously agreed that there might be an execution policy attached to a `scheduler`. The question that comes to mind is what should we do if we have execution policy passed via `bulk` parameters that conflicts with execution policy attached to a scheduler?

This is another area to explore. It's quite obvious that execution policy passed to `bulk` describes possible parallelization ways of `bulk` callable object, while it's not 100% obvious what execution policy attached to a scheduler means outside of [P2500R2] proposal.

We might choose different strategies (not all of them are mutually exclusive):

- Reconsider the decisions we made for [P2500R2] by stopping attaching policies to schedulers.
- Reduce policies: choose the most conservative one. For example, for the passed policy `par` and attached policy `seq` we reduce it the resulting policy to `seq` because it's the most conservative one.

    - For "peer" conflicting policies: `par` and `unseq` we might either want to reduce it to `seq` or might want to give a compile-time error.
- Give a compile-time error right away for any pair of conflicting policies.

Anyway, exploration of this problem is out of scope of this paper. For the purpose of this paper, we need to ensure that the way the given function is called is compatible with the execution policy passed to `bulk`.

§ 4.5. Expectations on customizations

The current specification of `bulk` in [exec.bulk] doesn't define any constraints/expectations for the customization of the algorithm. The default implementation (a serial version of `bulk`) is expected to be very different that its customizations.

Having customizations in mind, we need to define the minimal expectations for calling `bulk`. We propose the following:

- Mandate `f` to be copy-constructible.
- Allow `f` to be invoked according to the specified execution policy (and don't assume it's going to be called serially).
- Allow the values produced by the previous sender to be decay copied (if the values produced are copyable).
- Allow the algorithm to handle cancellation requests, but don't mandate it.

We want to require that the given functor be copy-constructible so that `bulk` requires the same type of function as a `for_each` with an execution policy. Adding an execution policy to `bulk`/`bulk_chunked` would also align them with `for_each`.

If `f` is allowed to be invoked concurrently, then implementations may want to copy the arguments passed to it.

Another important point for a `bulk` operation is how it should react to cancellation requests. We want to specify that `bulk` should be able to react to cancellation requests, but not make this mandatory. It shall be also possible for vendors to completely ignore cancellation requests (if, for example, supporting cancellation would actually slow down the main uses-cases that the vendor is optimizing for). Also, the way cancellation is implemented should be left unspecified.

Checking the cancellation token every iteration may be expensive for certain cases. A cancellation check requires an acquire memory barrier (which may be too much for really small functors), and

might prevent vectorization. Thus, implementations may want to check the cancellation every *N* iterations; *N* can be a statically known number, or can be dynamically determined.

As `bulk`, `bulk_chunked`, and `bulk_unchunked` are customization points, we need to specify these constraints to all three of them.

§ 4.6. Exception handling

While there are multiple solutions to specify which errors can be thrown by `bulk`, the most sensible ones seem to be:

1. pick an arbitrary exception thrown by one of the invocations of `f` (maybe using a atomic operations to select the first thrown exception),
2. reduce the exceptions to another exception that can carry one or more of the thrown exceptions using a user-defined reduction operation (similar to using `exception_list` as discussed by [P0333R0]),
3. allow implementations to produce a new exception type (e.g., to represent failures outside of `f`, or to represent failure to transmit the original exception to the receiver)

One should note that option 2 can be seen as a particular case of option 3.

Also, option 2 seems to be more complex than option 1, without providing any palpable benefits to the users.

The third option is very useful when implementations may fail outside of the calls to the given functor. Also, there may be cases in which exceptions cannot be transported from the place they were thrown to the place they need to be reported.

Based on the experience with the existing parallel frameworks we incline to recommend the option one because

- In a parallel execution the common behavior is non-deterministic by nature. Thus, we can peak arbitrary exception to throw
- Catching any exception already indicates that something went wrong, thus a good implementation might initiate a cancellation mechanism to finish already failed work as soon as possible.

On top of that, for special cases we also want to allow option 3.

§ 5. Specification

§ 5.1. In [execution.syn]

```
struct let_value_t { unspecified };
struct let_error_t { unspecified };
struct let_stopped_t { unspecified };
struct bulk_t { unspecified };
```

```
struct bulk_chunked_t { unspecified };
struct bulk_unchunked_t { unspecified };
```

[...]

```
inline constexpr let_value_t let_value{};
inline constexpr let_error_t let_error{};
inline constexpr let_stopped_t let_stopped{};
inline constexpr bulk_t bulk{};
```

```
inline constexpr bulk_chunked_t bulk_chunked{};
inline constexpr bulk_unchunked_t bulk_unchunked{};
```

§ 5.2. In [exec.bulk]

[Editorial note: Rename the title of the section from “execution::bulk” to “execution::bulk, execution::bulk_chunked, and execution::bulk_unchunked”. *— end note]*

[Editorial note: Apply the following changes (no track changes for paragraph numbering): *— end note]*

1. `bulk` , `bulk_chunked`, and `bulk_unchunked` run ~~s~~ a task repeatedly for every index in an index space.

2. The ~~name~~ names `bulk` , `bulk_chunked`, and `bulk_unchunked` ~~denotes a~~ denote pipeable sender adaptor ~~object~~ objects . Let *`bulk-algo`* be either `bulk`, `bulk_chunked`, or `bulk_unchunked`. For subexpressions `sndr`, `policy`, `shape`, and `f`, let `Policy` be `remove_cvref_t<decltype(policy)>`, `Shape` be `decltype(auto(shape))` , and `Func` be `decay_t<decltype((f))>` . If

- `decltype((sndr))` does not satisfy `sender`, or
- `is_execution_policy_v<Policy>` is `false`, or
- `Shape` does not satisfy `integral`, or
- ~~`decltype((f))`~~ `Func` does not ~~satisfy *movable-value*~~ model `copy_constructible`

,

~~bulk(sndr, shape, f)~~ *`bulk-algo`*`(sndr, policy, shape, f)` is ill-formed.

3. Otherwise, the expression ~~`bulk(sndr, shape, f)`~~ *`bulk-algo`*`(sndr, policy, shape, f)` is expression-equivalent to:

```
transform_sender(
  get-domain-early(sndr),
  make-sender(bulkbulk-algo, product-type<see below, Shape, Func>{polic
```

except that `sndr` is evaluated only once.

3.1. The first template argument of *`product-type`* is `Policy` if `Policy` models `copy_constructible`, and `const Policy&` otherwise.

4. Let `sndr` and `env` be subexpressions such that `Sndr` is `decltype((sndr))`. If *`sender-for`*`<Sndr, bulk_t>` is `false`, then the expression `bulk.transform_sender(sndr, env)` is ill-formed; otherwise, it is equivalent to:

```
auto [_, data, child] = sndr;
auto& [policy, shape, f] = data;
auto new_f = [func=std::move(f)](Shape begin, Shape end, auto&&... v
    noexcept(noexcept(f(begin, vs...))) {
  while (begin != end) func(begin++, vs...);
}
return bulk_chunked(std::move(child), policy, shape, std::move(new_f
```

[Note ?: This causes the `bulk(sndr, policy, shape, f)` sender to be expressed in terms of `bulk_chunked(sndr, policy, shape, f)` when it is connected to a receiver whose execution domain does not customize `bulk`. — *end note]*

5. The exposition-only class template `impls-for` ([exec.snd.general]) is specialized for `bulk_chunked_t` as follows:

```
namespace std::execution {
  template<>
  struct impls-for<bulk_chunked_t> : default-impls {
    static constexpr auto complete = see below;
  };
}
```

5.1. The member `impls-for<bulk_chunked_t>::complete` is initialized with a callable object equivalent to the following lambda:

```
[]<class Index, class State, class Rcvr, class Tag, class... Args>
  (Index, State& state, Rcvr& rcvr, Tag, Args&&... args) noexcept
  -> void requires see below {
    if constexpr (same_as<Tag, set_value_t>) {
      auto& [policy, shape, f] = state;
      constexpr bool nothrow = noexcept(f(auto(shape), auto(shape),
      TRY-EVAL(rcvr, [&]() noexcept(nothrow) {
        f(static_cast<decltype(auto(shape))>(0), auto(shape), args..
        Tag()(std::move(rcvr), std::forward<Args>(args)...);
      }());
    } else {
      Tag()(std::move(rcvr), std::forward<Args>(args)...);
    }
  }
```

5.1.1. The expression in the *requires-clause* of the lambda above is `true` if and only if `Tag` denotes a type other than `set_value_t` or if the expression `f(auto(shape), auto(shape), args...)` is well-formed.

6. The exposition-only class template `impls-for` ([exec.snd.general]) is specialized for ~~`bulk_t`~~ `bulk_unchunked_t` as follows:

```
namespace std::execution {
  template<>
  struct impls-for<~~bulk_t~~bulk_unchunked_t> : default-impls {
```

```
    static constexpr auto complete = see below;
  };
}
```

6.1. The member *impls-for*<~~bulk_t~~bulk_unchunked_t>::*complete* is initialized with a callable object equivalent to the following lambda:

```
[]<class Index, class State, class Rcvr, class Tag, class... Args>
  (Index, State& state, Rcvr& rcvr, Tag, Args&&... args) noexcept
  -> void requires see below {
    if constexpr (same_as<Tag, set_value_t>) {
      auto& [shape, f] = state;
      constexpr bool nothrow = noexcept(f(auto(shape), args...));
      TRY-EVAL(rcvr, [&]() noexcept(nothrow) {
        for (decltype(auto(shape)) i = 0; i < shape; ++i) {
          f(auto(i), args...);
        }
        Tag()(std::move(rcvr), std::forward<Args>(args)...);
      }());
    } else {
      Tag()(std::move(rcvr), std::forward<Args>(args)...);
    }
  }
```

6.1.1. The expression in the *requires-clause* of the lambda above is `true` if and only if `Tag` denotes a type other than `set_value_t` or if the expression `f(auto(shape), args...)` is well-formed.

7. Let the subexpression `out_sndr` denote the result of the invocation ~~`bulk`~~ *bulk-algo* `(sndr, policy, shape, f)` or an object equal to such, and let the subexpression `rcvr` denote a receiver such that the expression `connect(out_sndr, rcvr)` is well-formed. The expression `connect(out_sndr, rcvr)` has undefined behavior unless it creates an asynchronous operation ([async.ops]) that, when started:

   - ~~on a value completion operation, invokes `f(i, args...)` for every `i` of type `Shape` from `0` to `shape`, where `args` is a pack of lvalue subexpressions referring to the value completion result datums of the input sender, and~~
   - ~~propagates all completion operations sent by `sndr`.~~
   - If `sndr` has a successful completion, where `args` is a pack of lvalue subexpressions referring to the value completion result datums of `sndr`, or decayed copies of those values if

they model `copy_constructible`, then:

- If `out_sndr` also completes successfully, then:
  - for `bulk`, invokes `f(i, args...)` for every `i` of type `Shape` from `0` to `shape`;
  - for `bulk_unchunked`, invokes `f(i, args...)` for every `i` of type `Shape` from `0` to `shape`;
    - *Recommended practice:* The underlying scheduler should execute each iteration on a distinct execution agent.
  - for `bulk_chunked`, invokes `f(b, e, args...)` zero or more times with pairs of `b` and `e` of type `Shape` in range `[0, shape]`, such that `b < e` and for every `i` of type `Shape` from `0` to `shape`, there is exactly one invocation with a pair `b` and `e`, such that `i` is in the range `[b, e)`.
- If `out_sndr` completes with `set_error(rcvr, eptr)`, then the asynchronous operation may invoke a subset of the invocations of `f` before the error completion handler is called, and `eptr` is an `exception_ptr` containing either:
  - an exception thrown by an invocation of `f`, or
  - a `bad_alloc` exception if the implementation fails to allocate required resources, or
  - an exception derived from `runtime_error`.
- If `out_sndr` completes with `set_stopped(rcvr)`, then the asynchronous operation may invoke a subset of the invocations of `f` before the stopped completion handler.

- If `sndr` does not complete with `set_value`, then the completion is forwarded to `recv`.
- For *`bulk-algo`*, the parameter `policy` describes the manner in which the execution of the asynchronous operations corresponding to these algorithms may be parallelized and the manner in which they apply `f`. Permissions and requirements on parallel algorithm element access functions ([algorithms.parallel.exec]) apply to `f`.

8. *[Note:* The asynchronous operation corresponding to *`bulk-algo`*`(sndr, policy, shape, f)` can complete with `set_stopped` if cancellation is requested or ignore cancellation requests. — *end note]*

6. Acknowledgements

Special thanks to Christian Trott and Tom Scogland for a very productive discussion on `bulk_unchunked` and help on working improvements.

7. Polls

7.1. SG1, Hagenberg, Austria, 2025

Summary: Forward P3481R1 with the following notes:

- `bulk_unchunked` should not have an execution policy
- Calling an un-customized `bulk_unchunked` is ill-formed on a concurrent scheduler
- The next revision of this paper (before LWG) needs wording for its error model

SF	F	N	A	SA
8	2	2	0	0

Consensus

7.2. SG1, Wrocław, Poland, 2024

[P3481R0] was presented at the SG1 meeting in November 2024 in Wrocław, Poland. SG1 provided the following feedback in the form of polls:

1. If we have chunking, we need to expose a control on the chunk size.

SF	F	N	A	SA
1	2	3	1	1

   No consensus

2. We need a version of the `bulk` API that presents the chunk to the callable in order to implement the parallel algorithms

SF	F	N	A	SA
4	2	1	0	1

Consensus in favor

3. We need a version of the bulk API that creates an execution agent per iteration.

SF	F	N	A	SA
2	3	3	0	0

Unanimous consent

4. We believe / desire:

    1. Bulk needs an execution policy to describe the callable, IF the scheduler also has an execution policy (for debugging for example) then a conservative choice should be used (`seq` is more conservative than `par`)
    2. No SG1 concerns with the proposed exception handling
    3. No change to status quo on default implementation of bulk being serial
    4. No change to status quo on bulk having a predecessor
    5. Forms of bulk needed:

        - `bulk` -> `f(i)` -> `bulk_chunked`
        - `bulk_chunked` -> `f(b, e)`
        - `bulk_unchunked` -> `f(i)` "execution agent per iteration"

SF	F	N	A	SA
5	2	1	0	0

Unanimous consent

§ References

§ Informative References

[ALG.FOREACH]
ISO WG21. *Working Draft: Programming Languages — C++ -- For each*. URL: https://eel.is/c++draft/alg.foreach

[EXEC.BULK]
ISO WG21. *Working Draft: Programming Languages — C++ -- `execution::bulk`*. URL: https://eel.is/c++draft/exec.bulk

[P0333R0]

Bryce Lelbach. *Improving Parallel Algorithm Exception Handling*. 15 May 2016. URL: https://wg21.link/p0333r0

[P2300R10]

Eric Niebler, Michał Dominiak, Georgy Evtushenko, Lewis Baker, Lucian Radu Teodorescu, Lee Howes, Kirk Shoop, Michael Garland, Bryce Adelstein Lelbach. *`std::execution`*. 28 June 2024. URL: https://wg21.link/p2300r10

[P2500R2]

Ruslan Arutyunyan, Alexey Kukanov. *C++ parallel algorithms and P2300*. 15 October 2023. URL: https://wg21.link/p2500r2

[P3481R0]

Lucian Radu Teodorescu, Lewis Baker, Ruslan Arutyunyan. *Summarizing std::execution::bulk() issues*. 16 October 2024. URL: https://wg21.link/p3481r0

[P3564R0]

Mark Hoemmen, Bryce Adelstein Lelbach,Michael Garland. *Make the concurrent forward progress guarantee usable in `bulk`*. 13 January 2025. URL: https://wg21.link/p3564r0

[PARALLEL_FOR]

Intel. *Intel® oneAPI Threading Building Blocks Developer Guide and API Reference -- parallel_for*. URL: https://www.intel.com/content/www/us/en/docs/onetbb/developer-guide-api-reference/2021-6/parallel-for.html